Instructions regarding the Course Project

Each group will implement a vision-based task on the lab's myCobot 280 Jetson Nano, using the kinematics developed in class. The project carries **20% of the EE430 course grade**, plus a **4% bonus** for a deep-learning component.

The project should connect three things: a camera sees something, your algorithm decides what it means, and the arm acts on it through forward and inverse kinematics. Groups are already formed; each group submits one report and one code archive.

By the end of the project, your group should be able to:

- Command the arm in joint space and task space, and explain how the commanded pose relates to the forward and inverse kinematics studied in class (Spong, Hutchinson & Vidyasagar, Robot Modeling and Control).
- Build a working computer-vision pipeline that turns camera frames into targets or decisions for the arm.
- Work on an embedded robot remotely and professionally over SSH, keeping all code under your own control.
- Document the work as a short technical paper and defend it in a viva.

Evaluation breakdown

The 20% is split across four components, with the deep-learning bonus on top. Marks for components A and B are awarded on a live demonstration in the lab, not on the report alone.

Component	**What is assessed**	**Weight (% of course)**
A. Kinematics implementation	Forward and inverse kinematics for a defined task, using the built-in pymycobot library	4
B. Computer-vision algorithm	A vision-based game or tracking task that drives the arm	4
C. Report	Literature review, methodology, results, discussion	8
D. Viva	Individual understanding of the whole project	4
Total		**20**
E. Bonus: deep learning	A learned model replacing or extending the classical vision pipeline	+4

The bonus is only considered if components A and B are working. A deep-learning model that runs but is not connected to the arm will earn at most half the bonus.

Working setup: connect over SSH, keep code on your own PC

All development must happen on your group's own laptop; the robot is only where the code runs. Connect to the myCobot's Jetson Nano over SSH, copy your code across for each run, and remove it at the end of your slot. The robot is shared by every group, so nothing of yours should live on it between sessions.

Technical components

A. Kinematics implementation (4%)

Define one concrete manipulation task and execute it with both forward and inverse kinematics through pymycobot. Examples: move to a set of marked points on the table, stack two blocks, trace a square in the horizontal plane, or pick-and-place between two fixed locations.

Marks (out of 4)	Criterion
1	**A1**: Task executes repeatably in the live demo
1	**A2**: FK verified against your own DH model, with an error table
1	**A3:** IK used correctly, with multiple-solution or failure case explained
1	**A4:** Clean frame and unit handling in code

B. Computer-vision algorithm (4%)

Build a camera-based task in which what the camera sees decides where the arm goes.

Marks (out of 4)	**Criterion**
1	**B1**: Robust detection under normal lab lighting
1	**B2**: Correct camera-to-robot calibration, explained
1	**B3**: Closed loop: vision output drives the arm using component A
1	**B4**: Quantitative evaluation over at least 10 trials

E. Bonus: deep learning (up to +4%)

Replace or extend the classical vision pipeline with a learned model. Examples: a YOLO-family detector for objects, a small CNN classifier for game states, or a hand-landmark model for gesture-commanded motion.

- Train or fine-tune on data you collect in the lab or justify clearly why a pretrained model suffices.

- Report accuracy on a held-out set, inference time per frame on the Jetson Nano (or your laptop, stated clearly), and a comparison against your classical pipeline from B.
- The model must drive the arm in the live demo. A model that runs but is not connected to the arm earns at most +2%.

C. Report (8%)

The report is a short technical paper of 8–12 pages (excluding appendices), in IEEE conference format, submitted as a PDF. Write it so that another group could reproduce your results from it.

Required sections:

1. Abstract (150–200 words): task, approach, key quantitative result.
2. Introduction: the problem, why it matters, and your contributions.
3. Literature review: at least 6 cited sources, at least 3 of them journal or conference papers. Compare the approaches, don't just list them.
4. System description: hardware setup, camera placement, software architecture diagram, and the SSH-based development workflow.
5. Methodology:
   ◦ Kinematic model: your DH frame assignment and parameter table for the myCobot 280 (Spong et al. convention), the resulting link transforms, and your FK/IK approach.
   ◦ Vision pipeline and camera-to-robot calibration.
   ◦ Deep-learning model, if attempted: architecture, dataset, training.
6. Results: FK error table, vision success rate and positioning error, photos or frame sequences of the task, and bonus metrics if applicable.
7. Discussion: failure cases, limitations, and what you would change.
8. Conclusion.
9. References (IEEE style).
10. Appendix: contribution statement listing what each member did, e-signed by all members.

Marks (out of 8)	**Criterion**
1.5	**C1**: Literature review: relevance, depth, comparison
2.5	**C2**: Methodology: correctness and clarity of kinematics and vision
2	**C3**: Results: quantitative, honest, well presented
1	**C4**: Discussion of limitations and failure cases
1	**C5**: Writing quality, figures, referencing

D. Viva (4%)

The viva is marked **individually**: each member receives their own score out of 4, even though the rest of the project is a group mark. Plan for about 10 minutes per member after a 5-minute group demonstration.

What to submit

Each group submits exactly two files, uploaded by one member on behalf of the group. Deadline and upload location: *Via Moodle*.

File	Name format	Contents
Report	EE430_GroupNN_Report.pdf	The full report described in section C
Code archive	EE430_GroupNN_Code.zip	All code, commented, in the structure below

Code archive structure

EE430_GroupNN_Code/

├── README.md # how to set up and run, step by step

├── requirements.txt # Python packages and versions

├── kinematics/ # component A: DH model, FK/IK scripts

├── vision/ # component B: detection, calibration

├── dl/ # bonus only: training and inference code

├── calibration/ # calibration data and results

├── main.py # runs the full demo task

└── results/ # logs, sample images, short demo video (≤ 2 min)

Do not include trained model weights larger than 50 MB or raw datasets; give a download link in the README instead.

Lab rules and academic integrity

Lab slots and robot issue/return

Groups may work on the robot in the Control Lab on Tuesday, Thursday or Friday, 2:00–4:30 PM. No robot work is permitted outside these slots.

Every session is logged through two Google Forms:

1. When you start (issuing the robot): fill the issue form and upload a photo of the robot before you begin. Link: https://docs.google.com/forms/d/e/1FAIpQLSey1iM6BeXusoOY03ul8VZLgzZ66-5N3vZPgKDeWj2z2ewwRw/viewform?usp=header

2. When you leave (returning the robot): fill the return form and upload a photo of the robot after you finish, once you have cleaned up your folder (Step 5 above) and returned the arm to its rest position. Link:

https://docs.google.com/forms/d/e/1FAIpQLSexfdwCZLsHVHYMwEku03JYmsw5W9f7Y4Nj-oLZBS0SiIxGCA/viewform?usp=header

Sessions without both forms will not count as lab time, and any damage found at the next issue will be traced to the last group that returned the robot.

Safety and robot care

- One group on the robot at a time.
- Keep one hand near the power switch whenever the arm moves under new code. Test new targets at low speed first.
- Keep hands, cables and faces out of the workspace while code is running.
- Never force a joint by hand while the servos are powered.
- Report any collision, unusual noise or overheating immediately and stop using the robot.

Academic integrity

- Vendor example code and open-source libraries may be used, but must be cited in the code header and the report. Your DH derivation, calibration and task logic must be your own.
- Sharing code between groups is not permitted. Similarity across groups will be checked.
- If you use AI tools (like I have used Claude here for drafting and finetuning these instructions :p), state in the report which parts they helped with. You must still be able to explain every line in the viva.

Resources

- myCobot 280 Jetson Nano user manual: official vendor documentation for this robot
- System functions, passwords, network and SSH setup: wired network sharing from a PC, VNC and SSH connection steps
- Python development guide for the 280 JN: pymycobot API, joint and coordinate control
- myCobot 280 kinematics and coordinate frames: vendor DH parameters; use only to cross-check your own derivation
- Spong, Hutchinson & Vidyasagar, *Robot Modeling and Control*: chapters on forward kinematics (DH convention) and inverse kinematics